

Transfer Learning

Agenda

- What is Transfer Learning?
- Methods for model tuning
- Parameter-Efficient Fine-tuning
 - Prompt-tuning
 - Adaptars
 - LoRa
 - BitFit

A large model for a small task

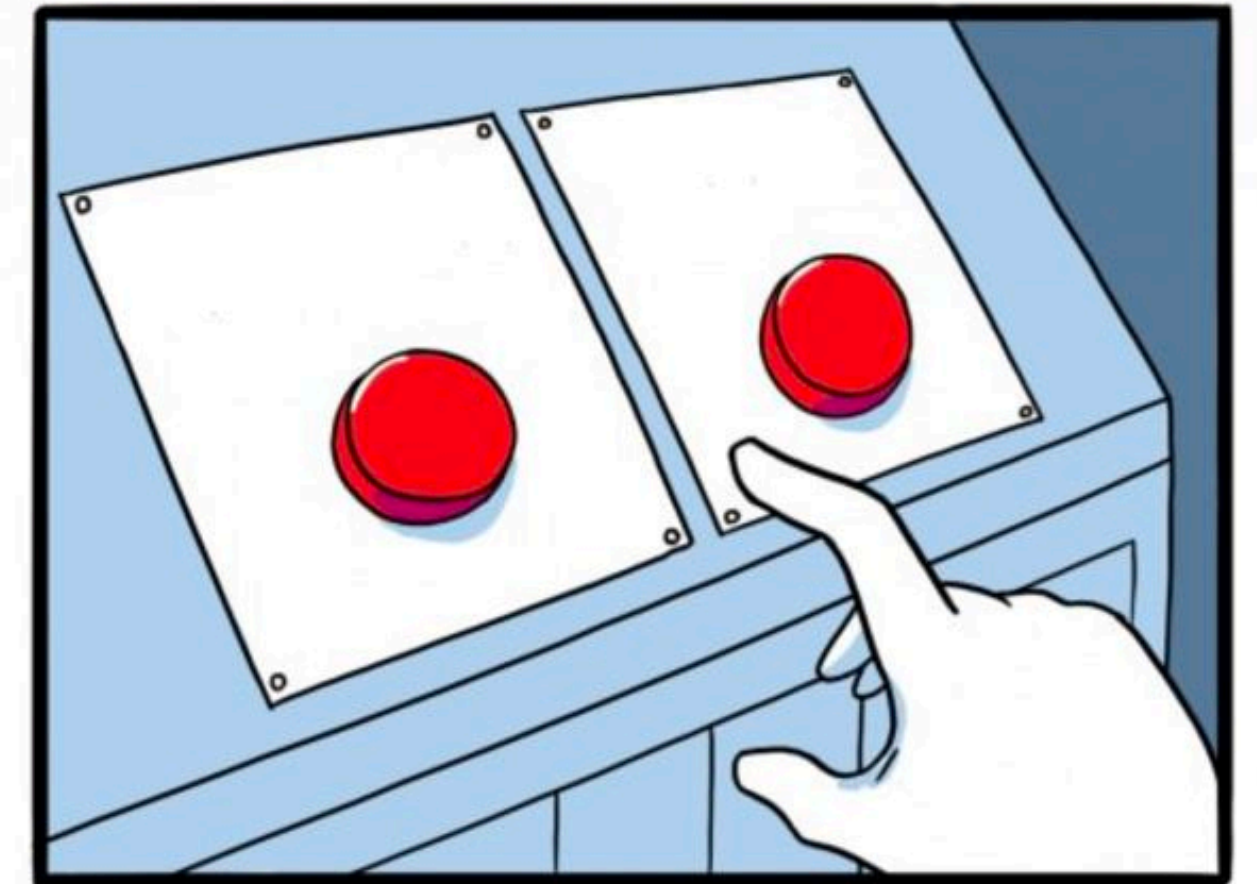


Large models can learn a lot of information
=> it's logical to use them for all tasks.



Large models require a lot of data to train.
They overfit on small tasks.

[How to solve this problem?]



JAKE-CLARK.TUMBLR

Transfer learning

Transfer learning – transferring the knowledge of a trained model to a new task.

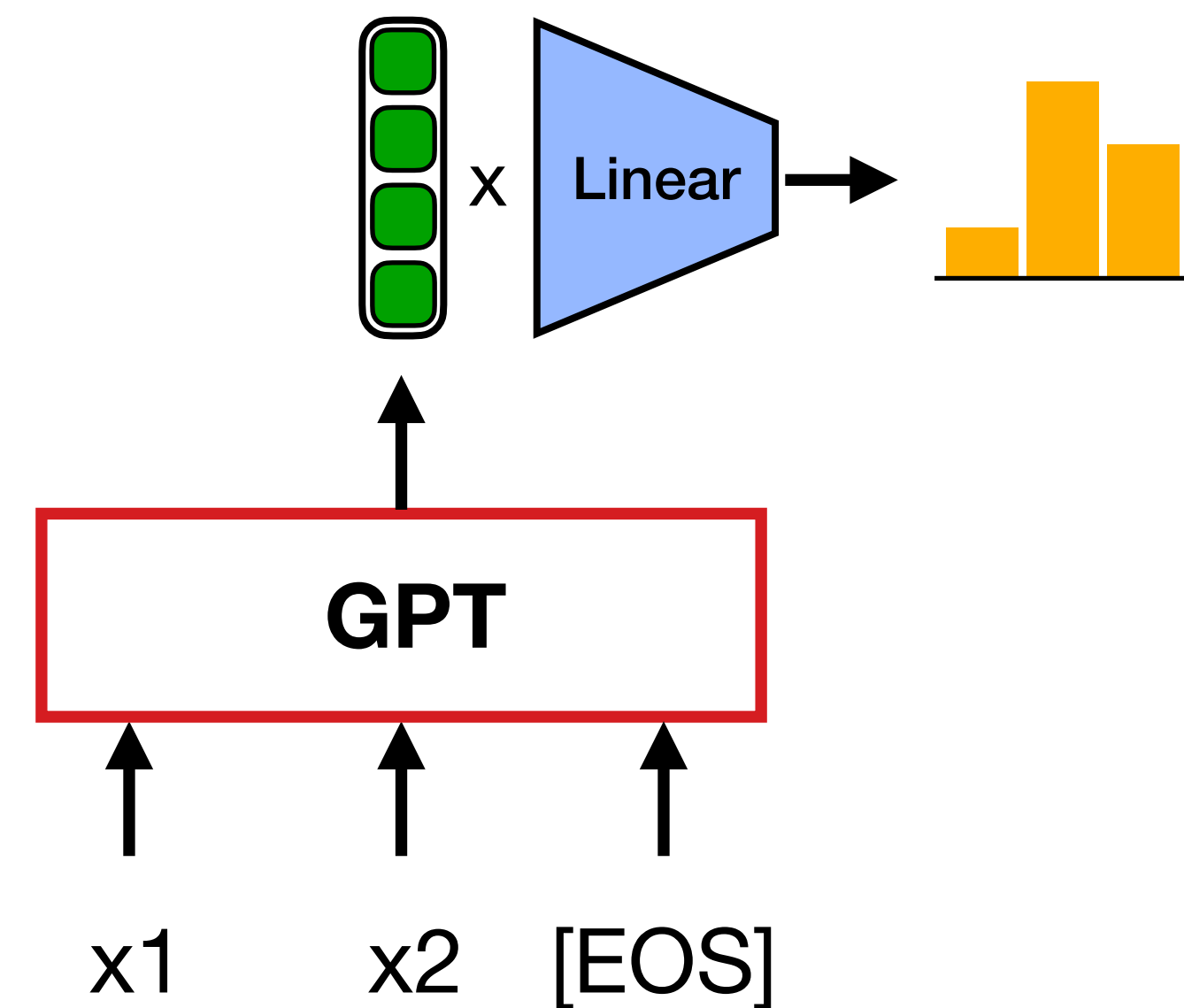
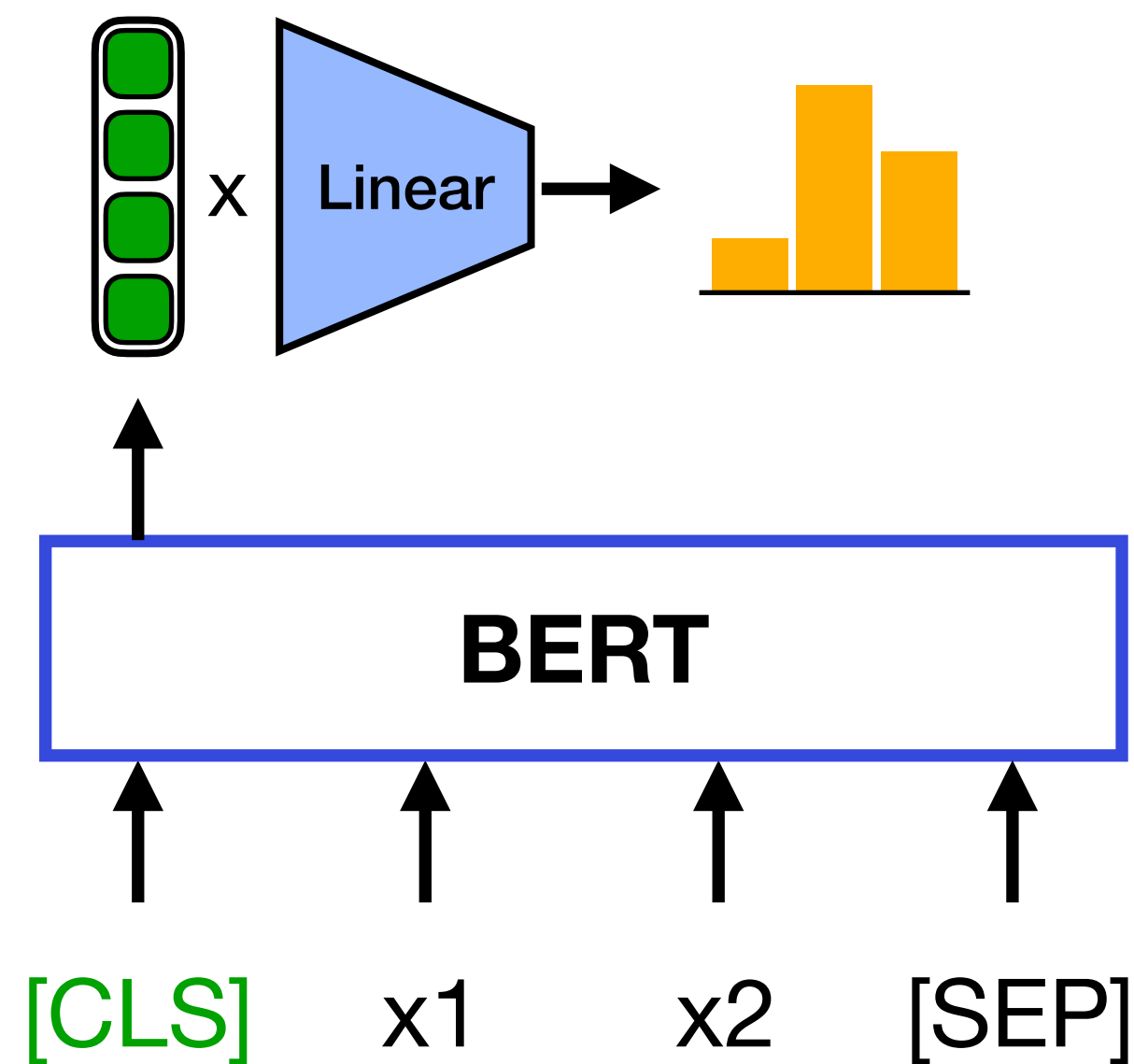
Training is divided into two stages:

- **Pre-training** – self-supervised training on a large data set
- **Fine-tuning** on a small *downstream* task

BERT and GPT modifications

Text classification:

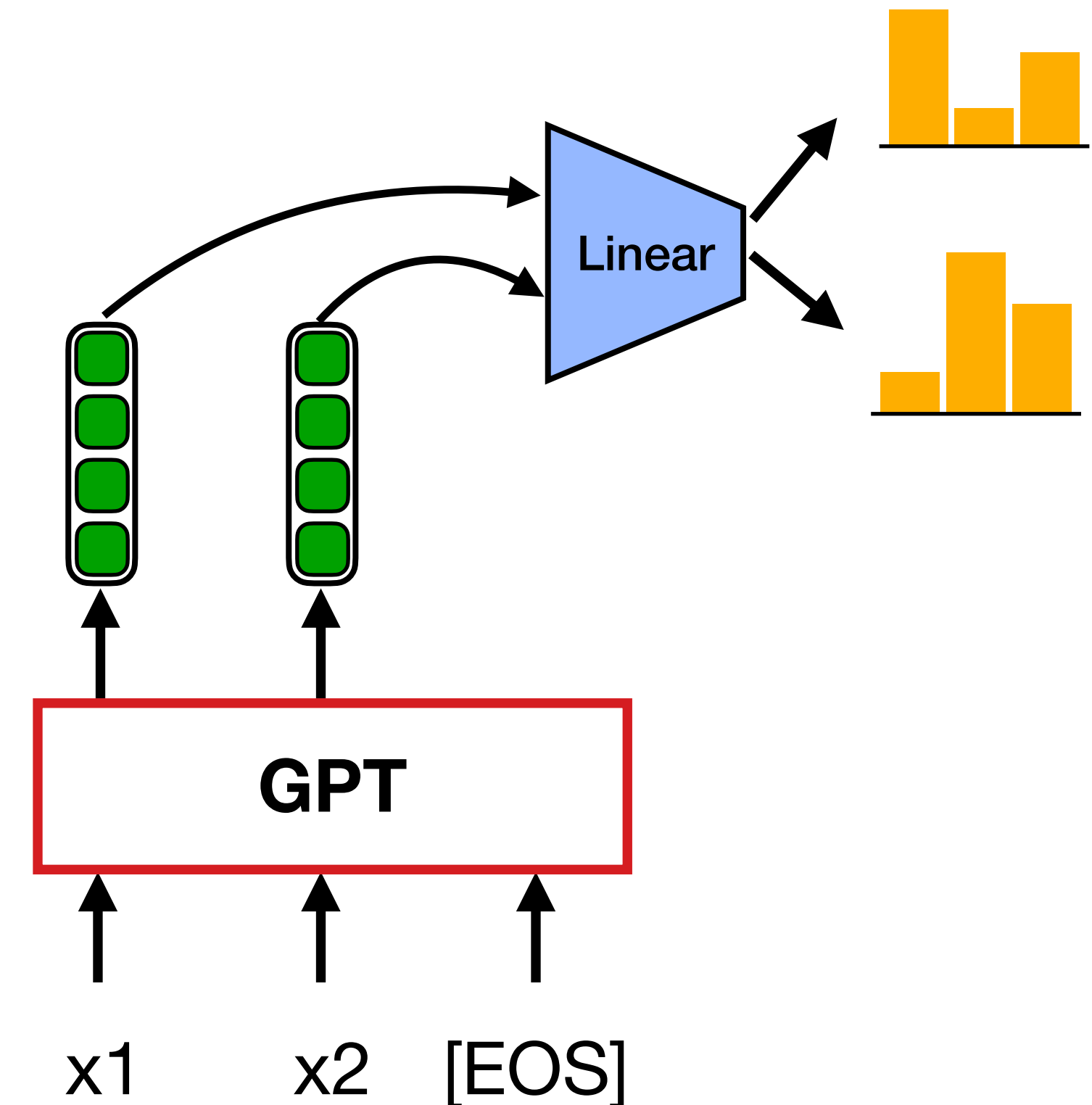
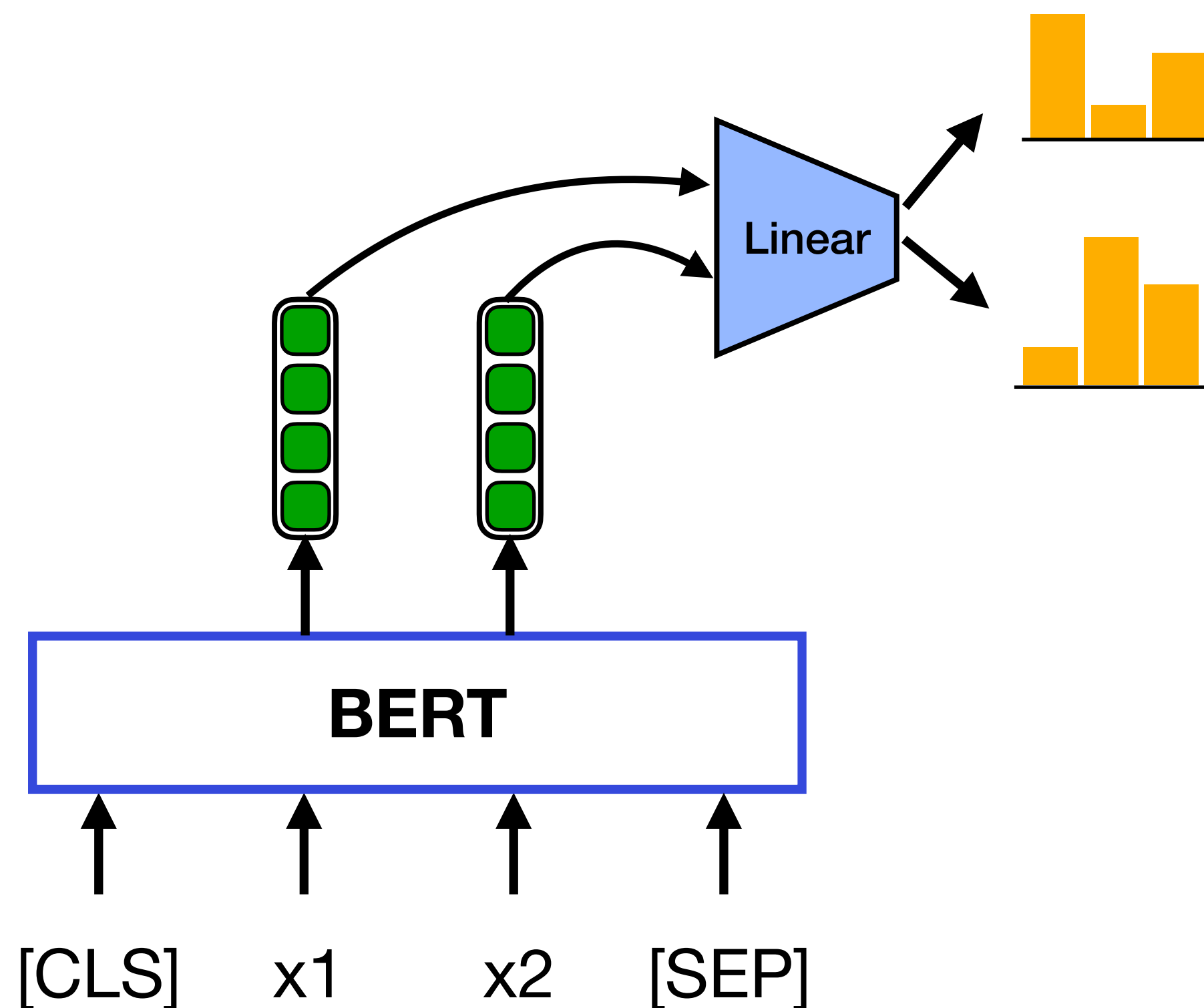
- Both models change the last linear layer.
- BERT takes the [CLS] token output.
- GPT takes the output of the last token.



BERT and GPT modifications

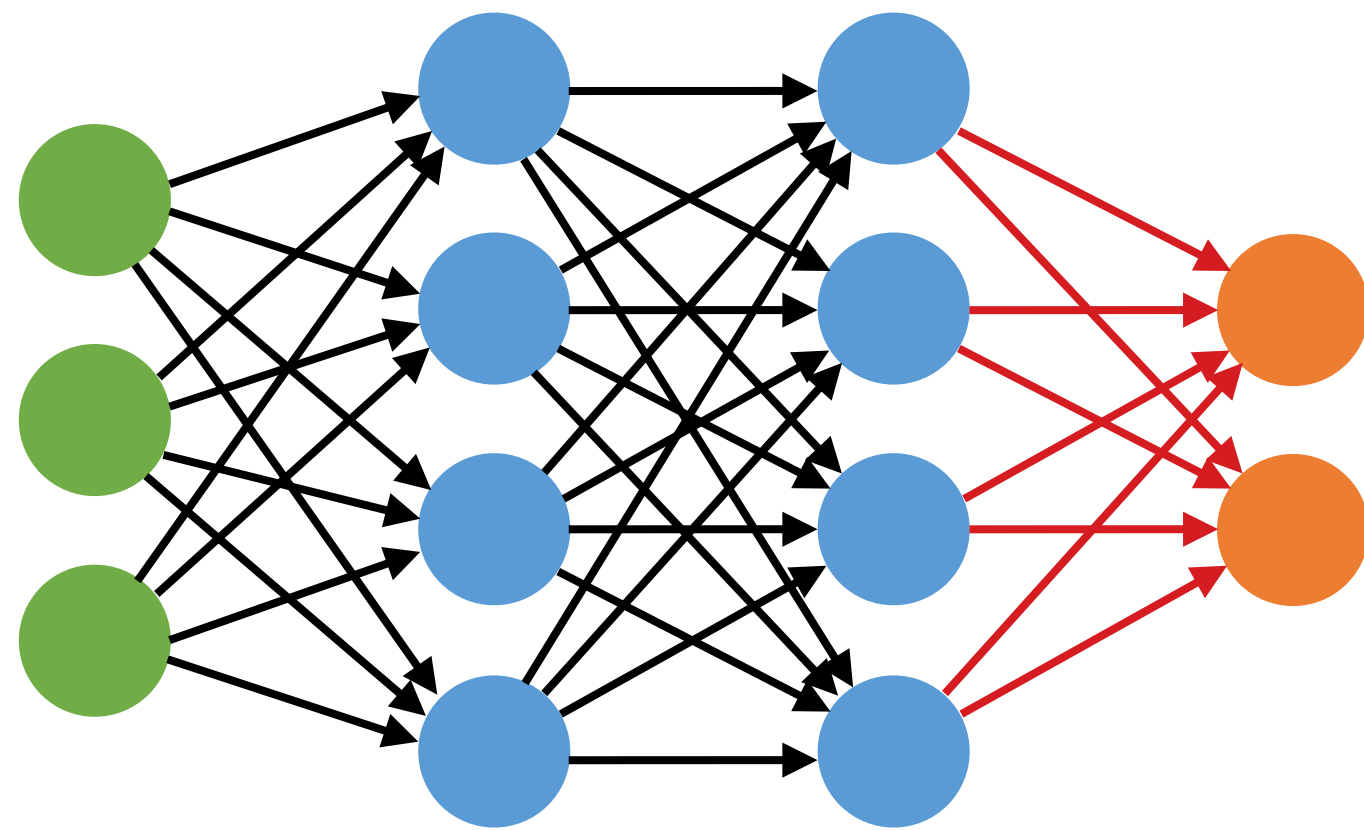
Token classification:

- The final linear layer is also modified.
- The outputs of each token are taken.
- It's better to use **BERT**.



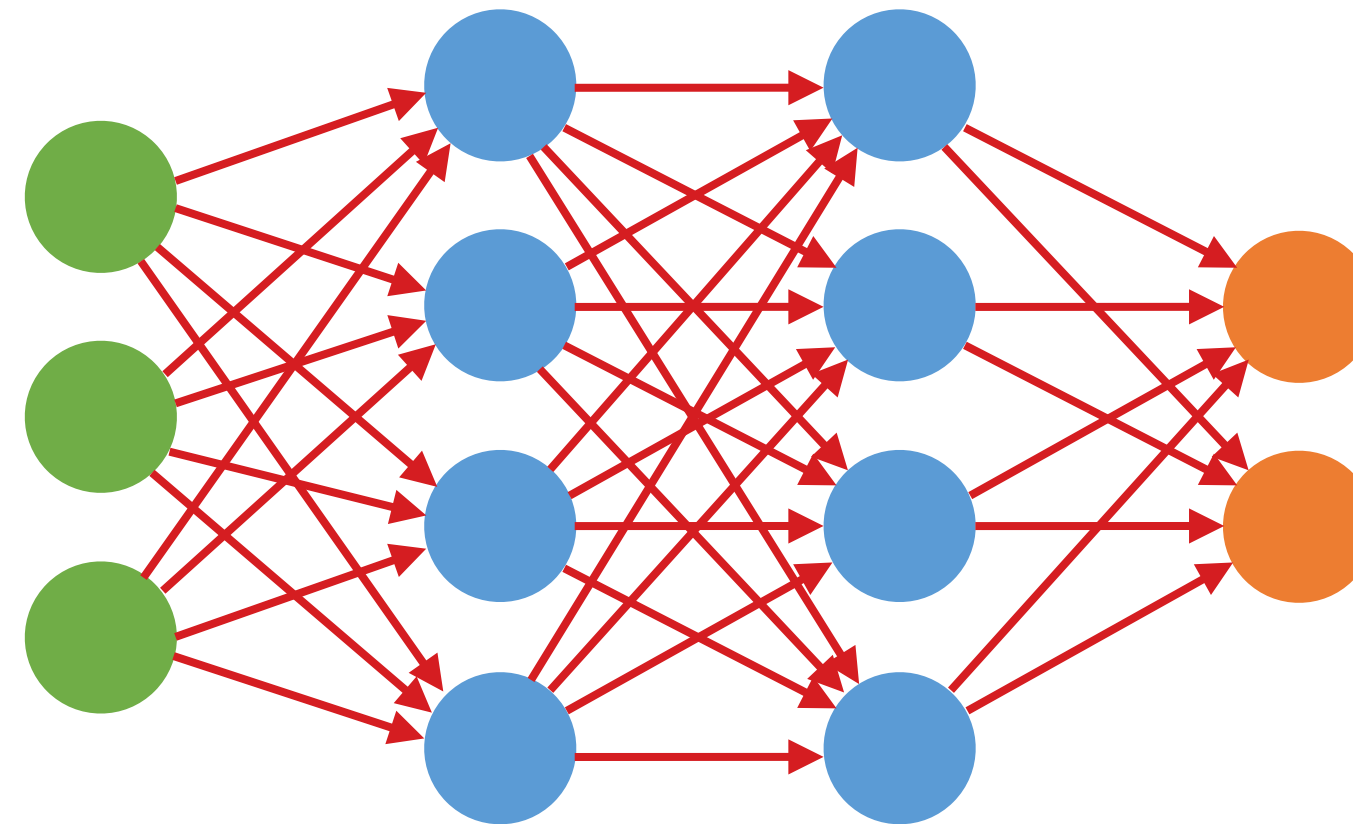
Model tuning methods

Head training
(linear probing)



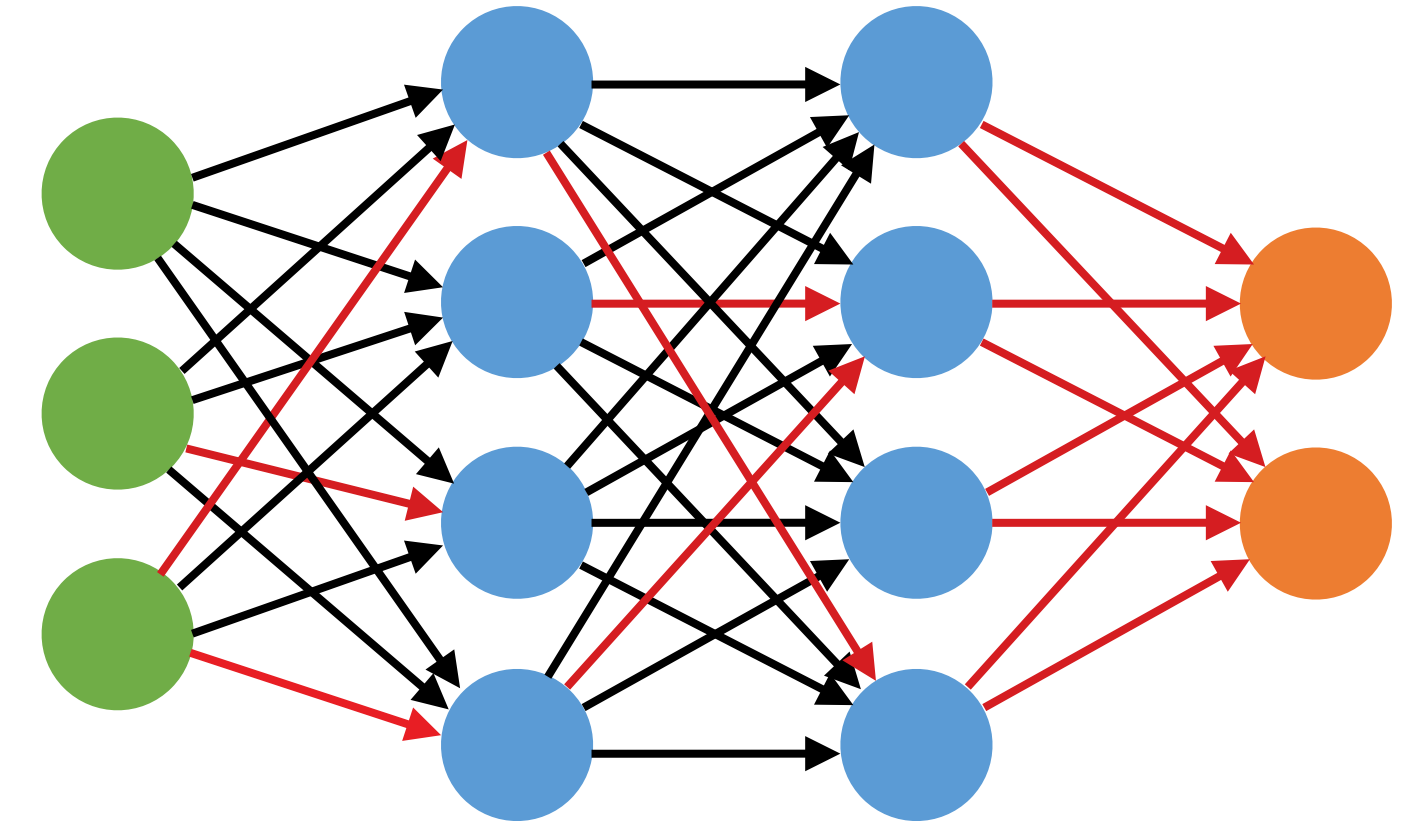
Only the last
layer is trained

Fine-tuning



The entire
model is trained

Parameter Efficient
Fine-tuning



A small set of
weights is trained

Probing vs Fine-tuning

A **downstream** task is a small data-intensive task for which the model is further trained (e.g., news classification).

Probing vs Fine-tuning

A **downstream** task is a small data-intensive task for which the model is further trained (e.g., news classification).

Probing

- The simplest and fastest method of further training
- Does not change the information extracted by the model

Fine-tuning

- The longest, but most effective method
- Trains the model to extract information important for the downstream task

Comparison in terms of ID and OOD

ID (In Distribution) – data within the downstream task distribution (the data we're training on)

- New Your Times

OOD (Out Of Distribution) – data outside the downstream task distribution (other data for a task of this type)

- The Washington Post
- Twitter posts

Comparison in terms of ID and OOD

ID (In Distribution) – data within the downstream task distribution (the data we're training on)

- New Your Times

OOD (Out Of Distribution) – data outside the downstream task distribution (other data for a task of this type)

- The Washington Post
- Twitter posts

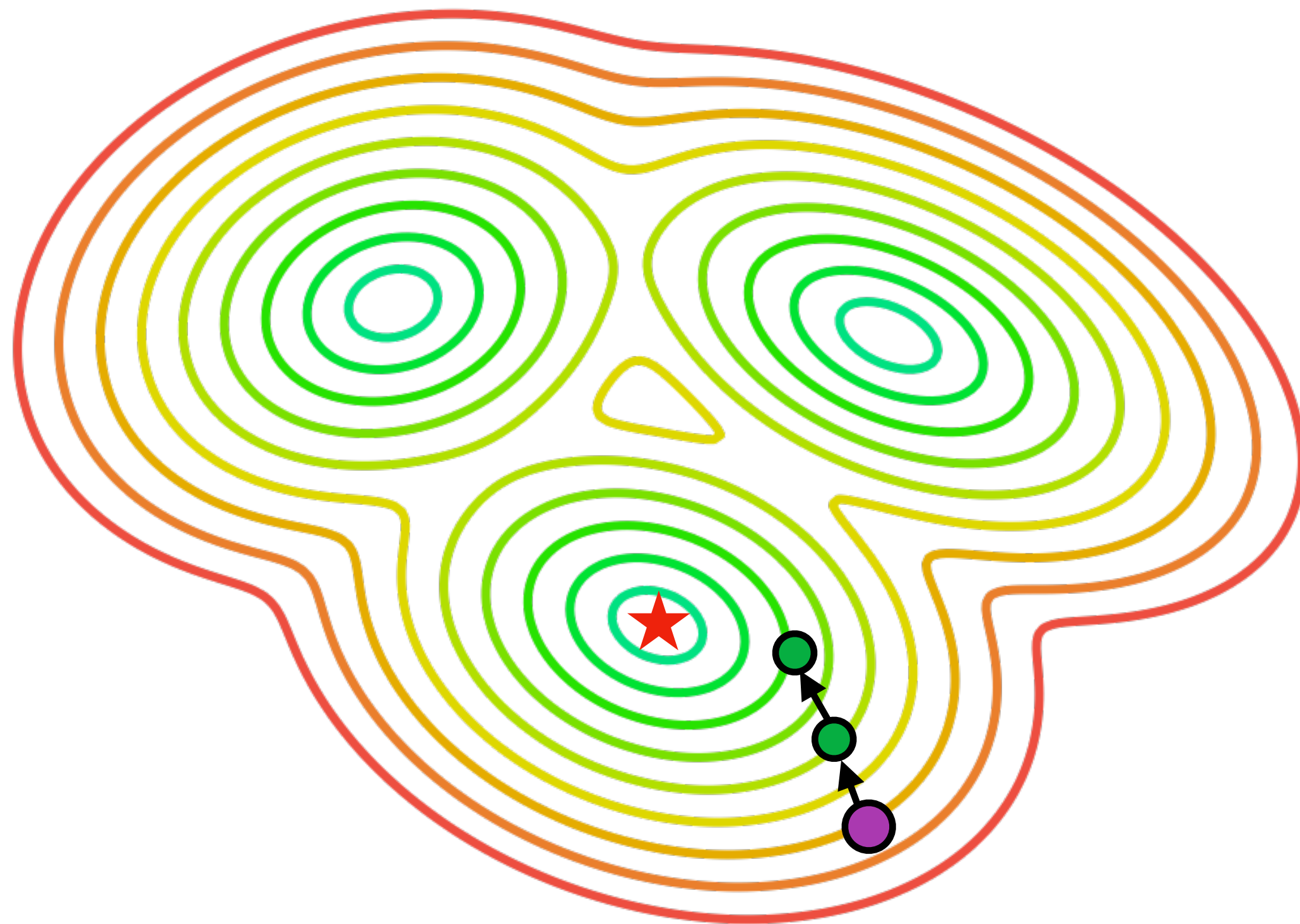
	Fine-tuning	Probing
ID test	85.1%	82.9%
OOD test	59.3%	66.2%

Classification accuracy

Fine-tuning spoils the model

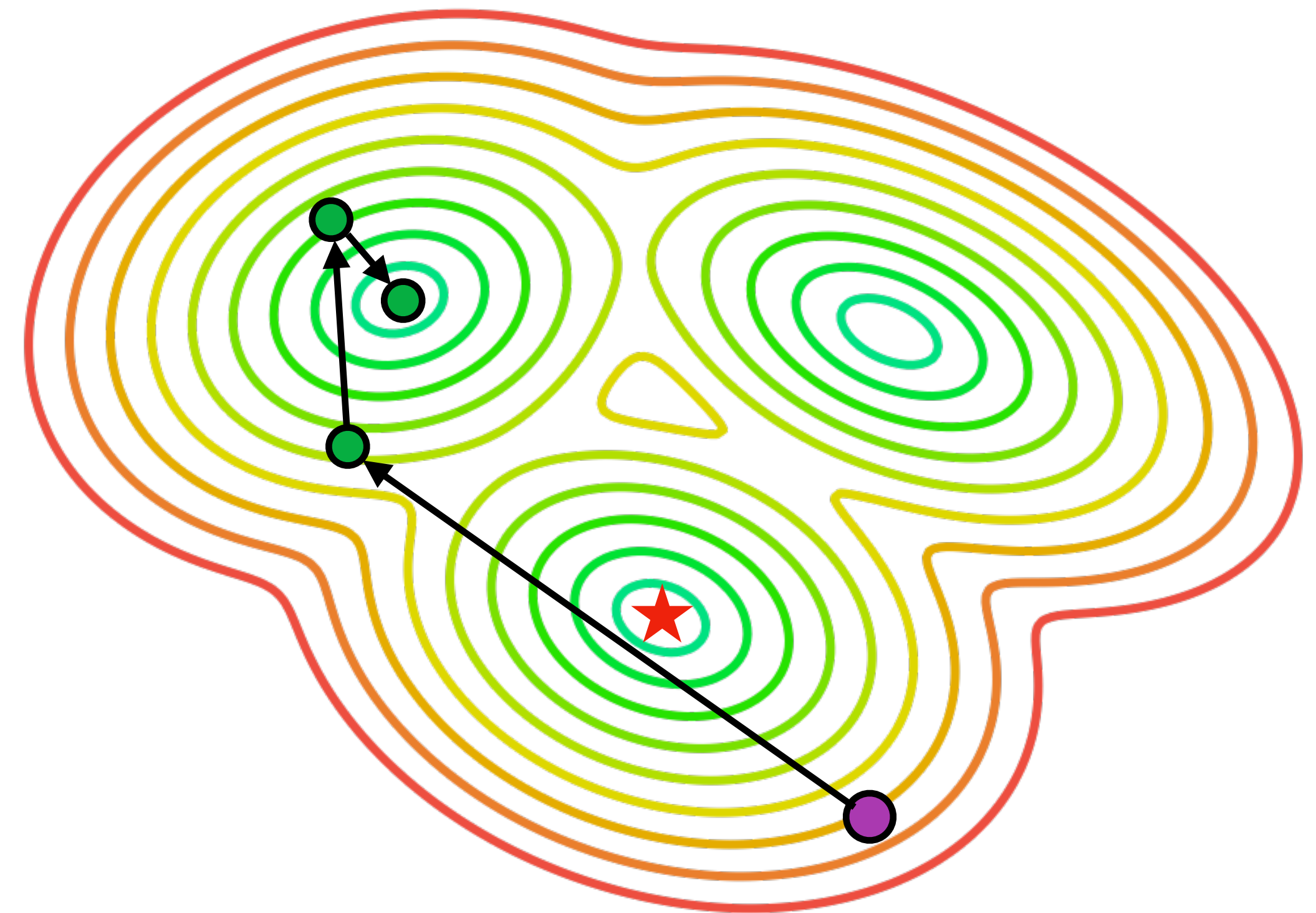
Due to random head initialization, gradients shift the model weights too much in the first iterations, leading to overfitting.

Probing



Not close enough to the optimum

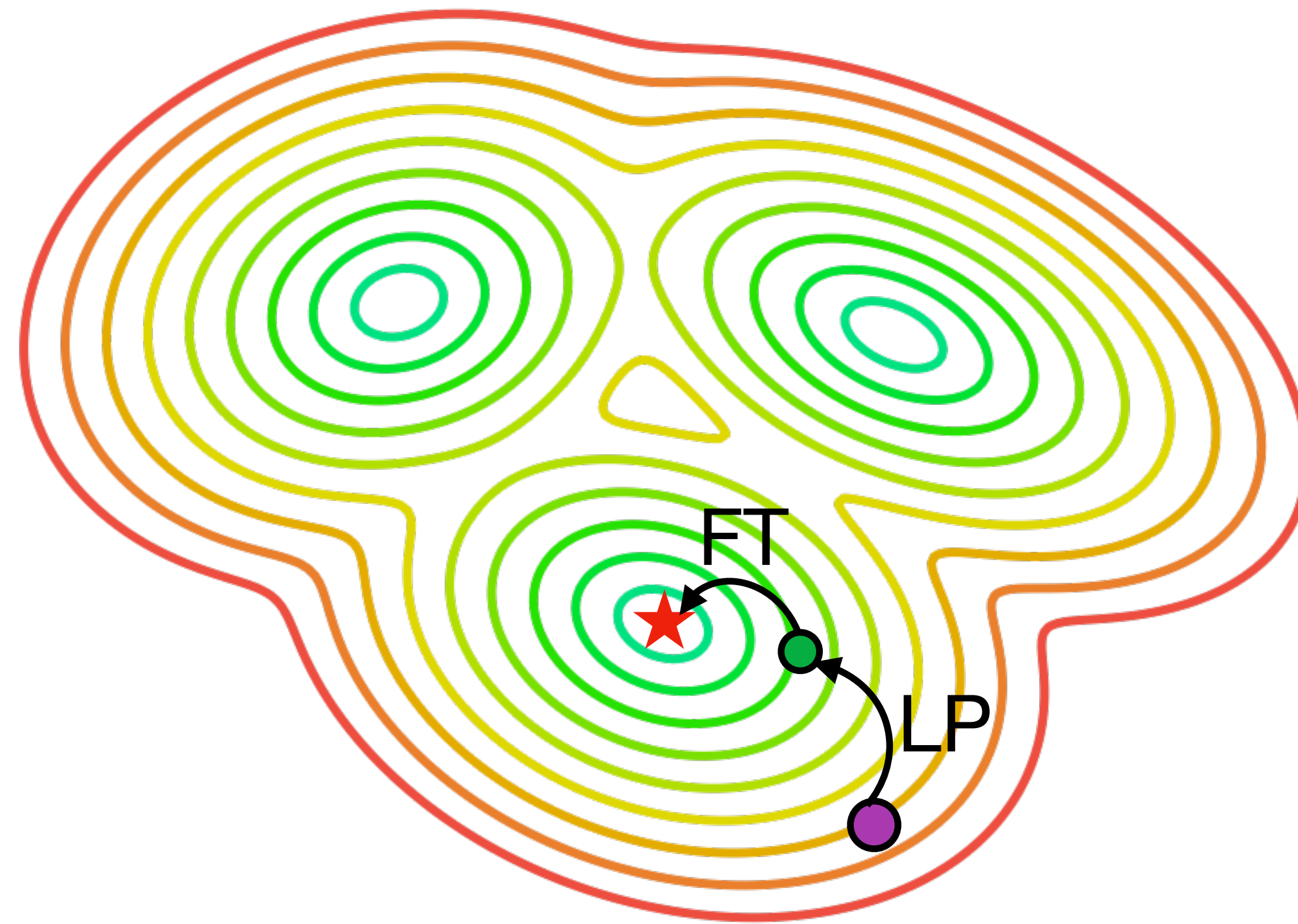
Fine-tuning



Close to the optimum, but the initial parameters of the model change significantly.

Probing → Fine-tuning

Let's first train the head, and then fine-tune the weights of the entire model.



Probing → Fine-tuning

Quality increases both on OOD and ID.

	Fine-tuning	Probing	Probing → Fine-tuning
ID test	85.1%	82.9%	85.7%
OOD test	59.3%	66.2%	68.9%

Classification accuracy

Agenda

- What is Transfer Learning?
- Methods for model tuning
- **Parameter-Efficient Fine-tuning**
 - Prompt-tuning
 - Adaptars
 - LoRa
 - BitFit

Parameter-Efficient Fine-tuning

- A method of pre-training models by changing a small number of parameters
- Used for training LLMs that are too costly to fine-tune
- When training on several different tasks, you can store only the set of changed parameters for each task

Few-shot and Zero-shot for GPT

- GPT-3 has been trained on 45 TB of data. This gives it useful properties.
- The model can be applied to a new problem by showing it examples of solutions in a prompt.

Translate from Russian to English:

стол => table

сыр => cheese

дом =>

It's called **Few-shot** learning.

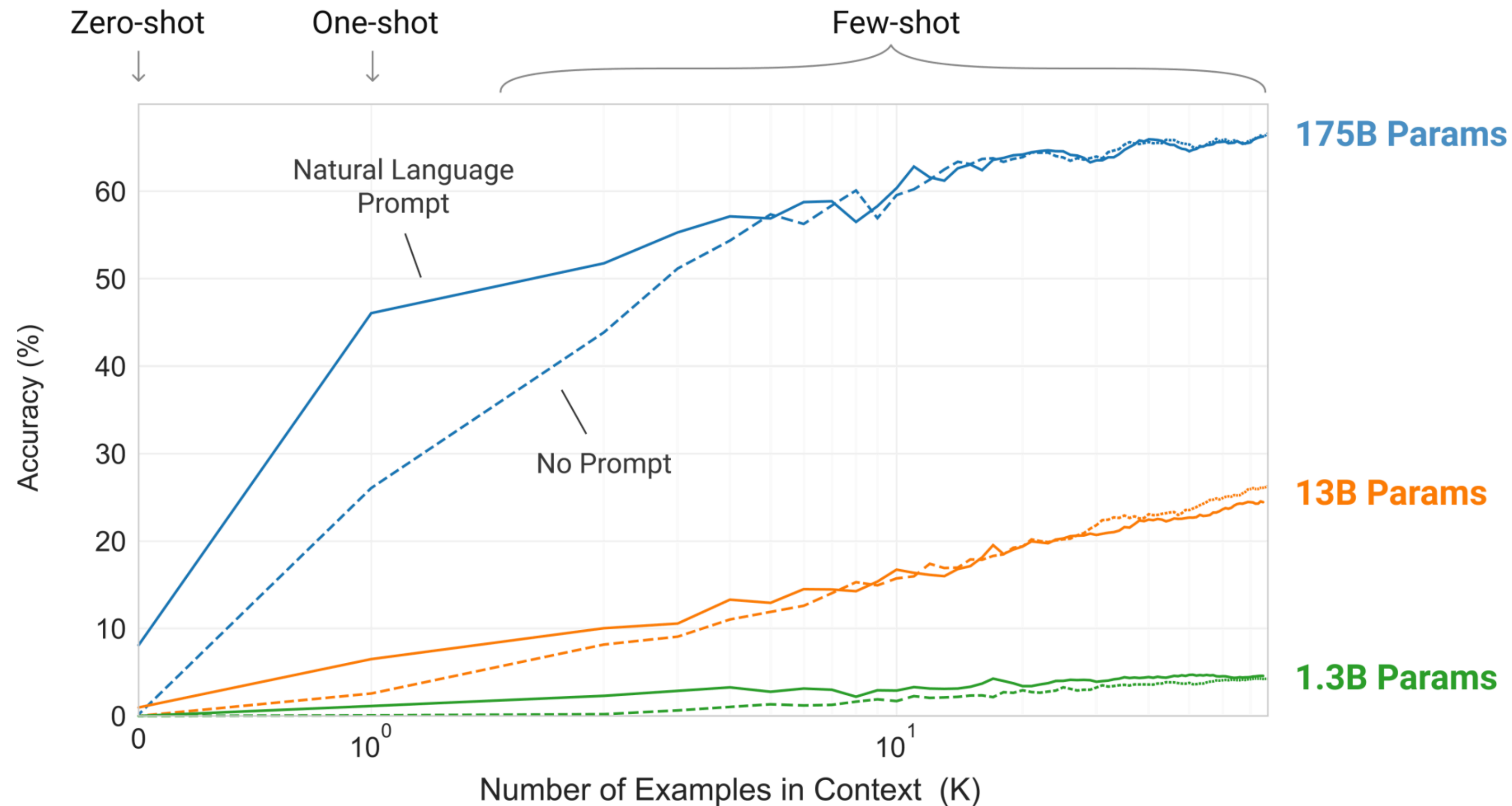
Translate from Russian to English:

дом =>

It's called **Zero-shot** learning.

Few-shot and Zero-shot for GPT

The larger the model, the better it performs.



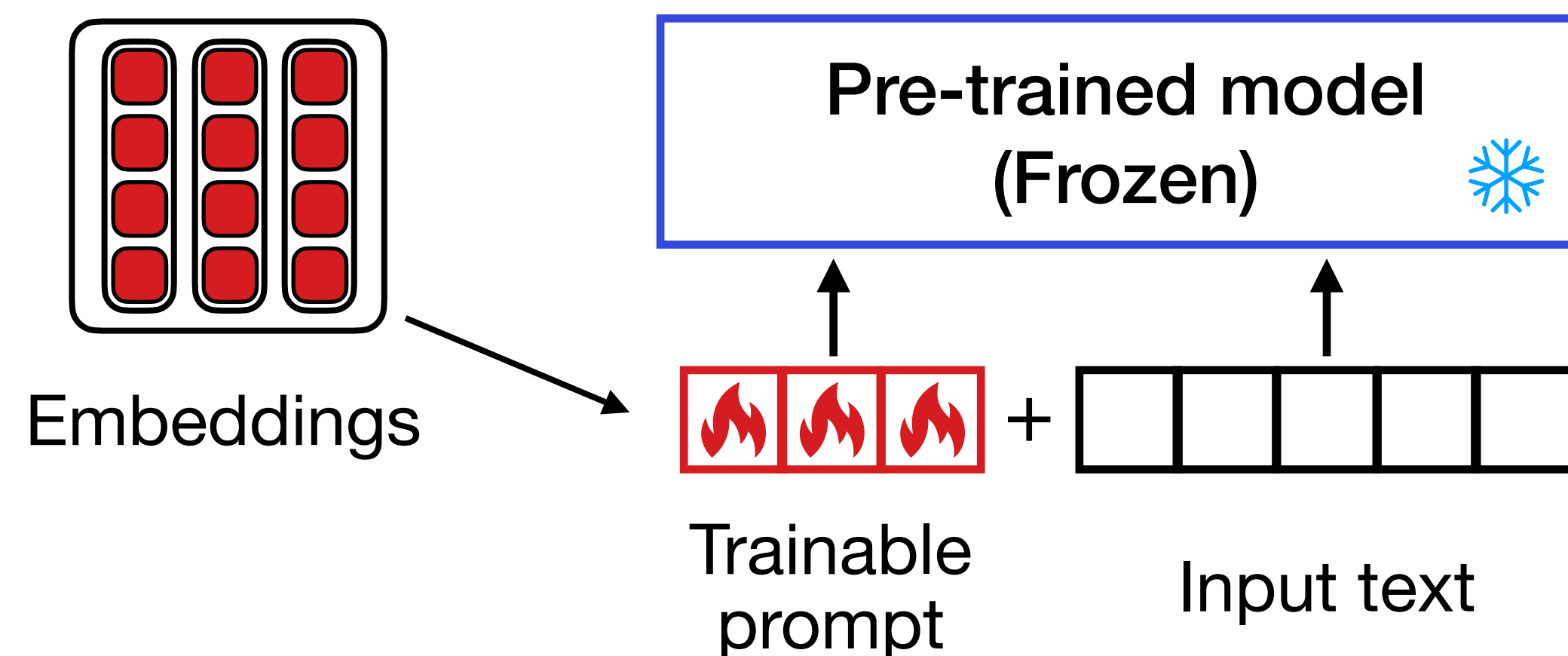
However, the result may be poor because of the fact that

- The model has not learnt how to solve this problem
- Prompt is not good enough

Prompt Tuning

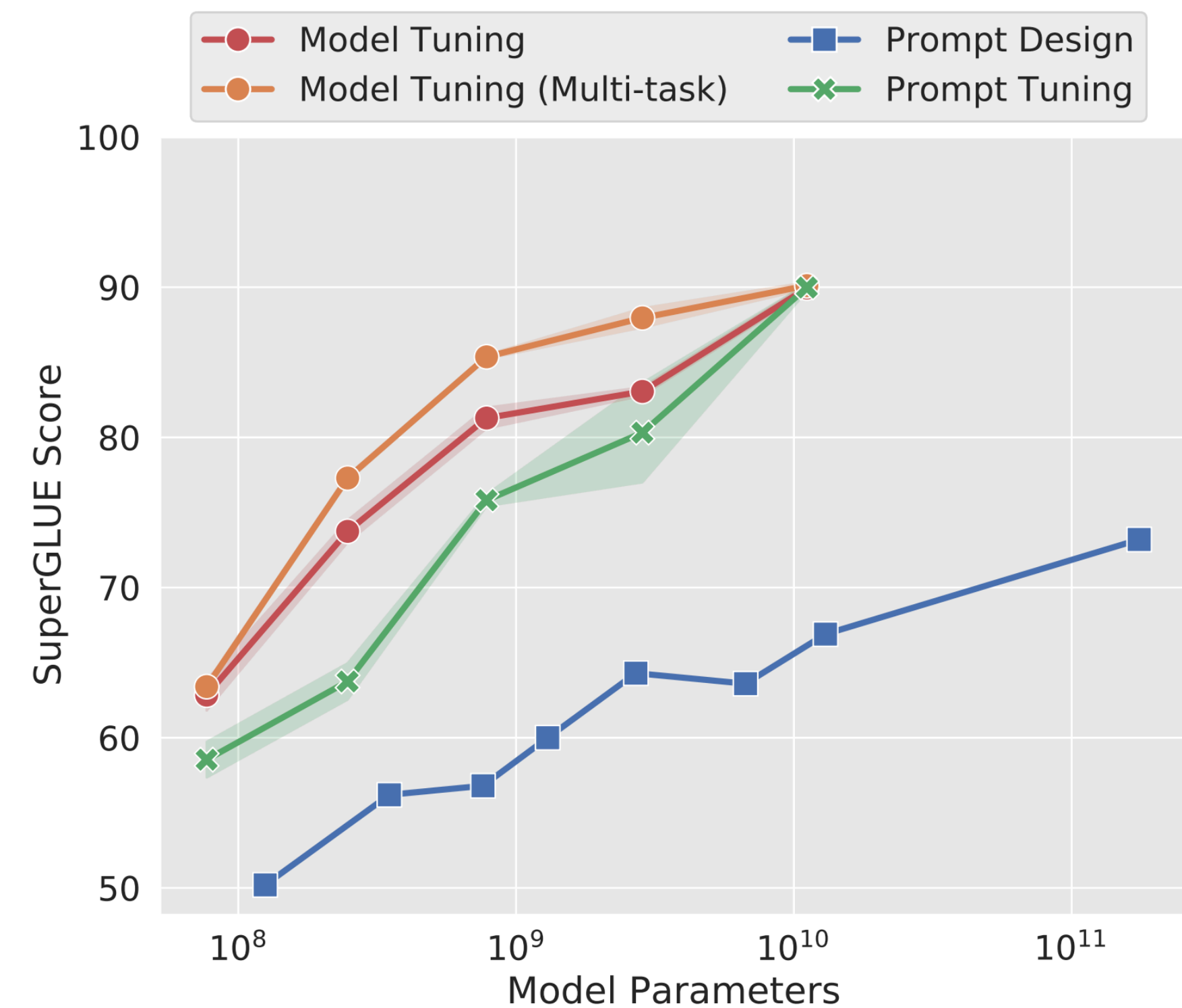
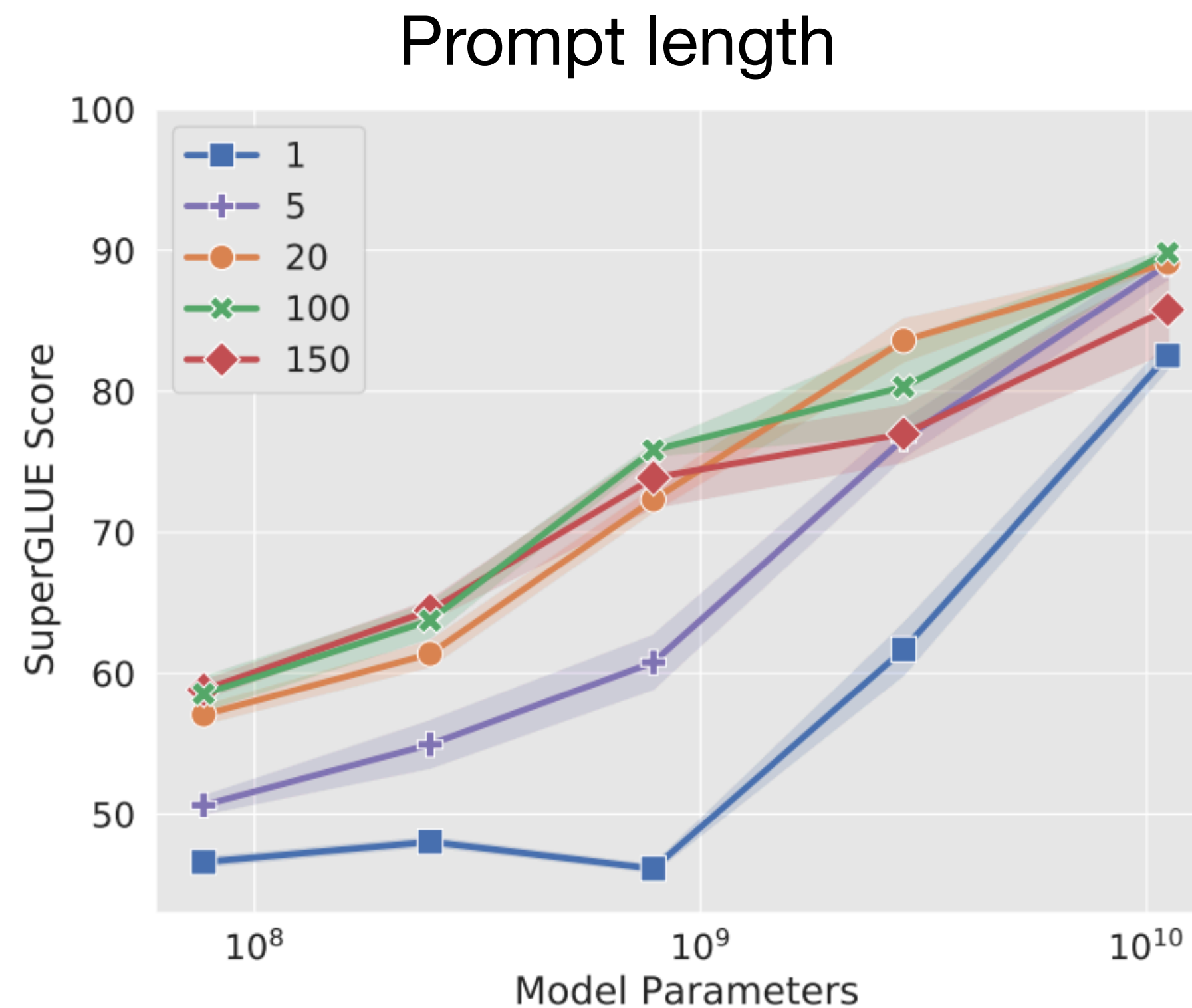
Idea: Let's try to automatically select the most appropriate prompt

- Initialise the prompt embeddings randomly, specifying only the number of them
- We can initialize the embeddings with embeddings of some prompt.
- For the classification task we need to train the head additionally



Prompt Tuning

- The length of the prompt directly affects the quality of the model
- However, the larger the model, the smaller the difference in quality

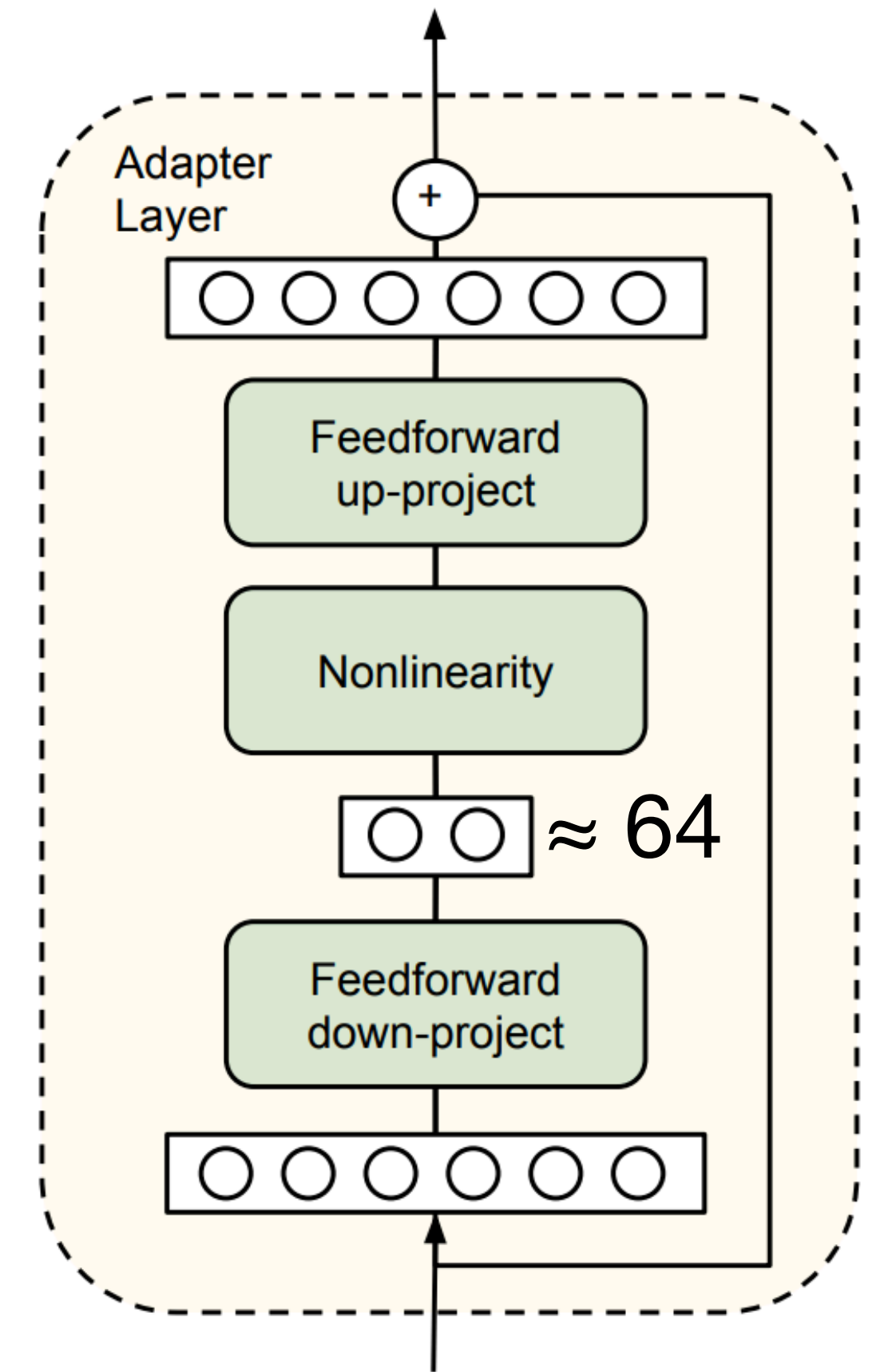
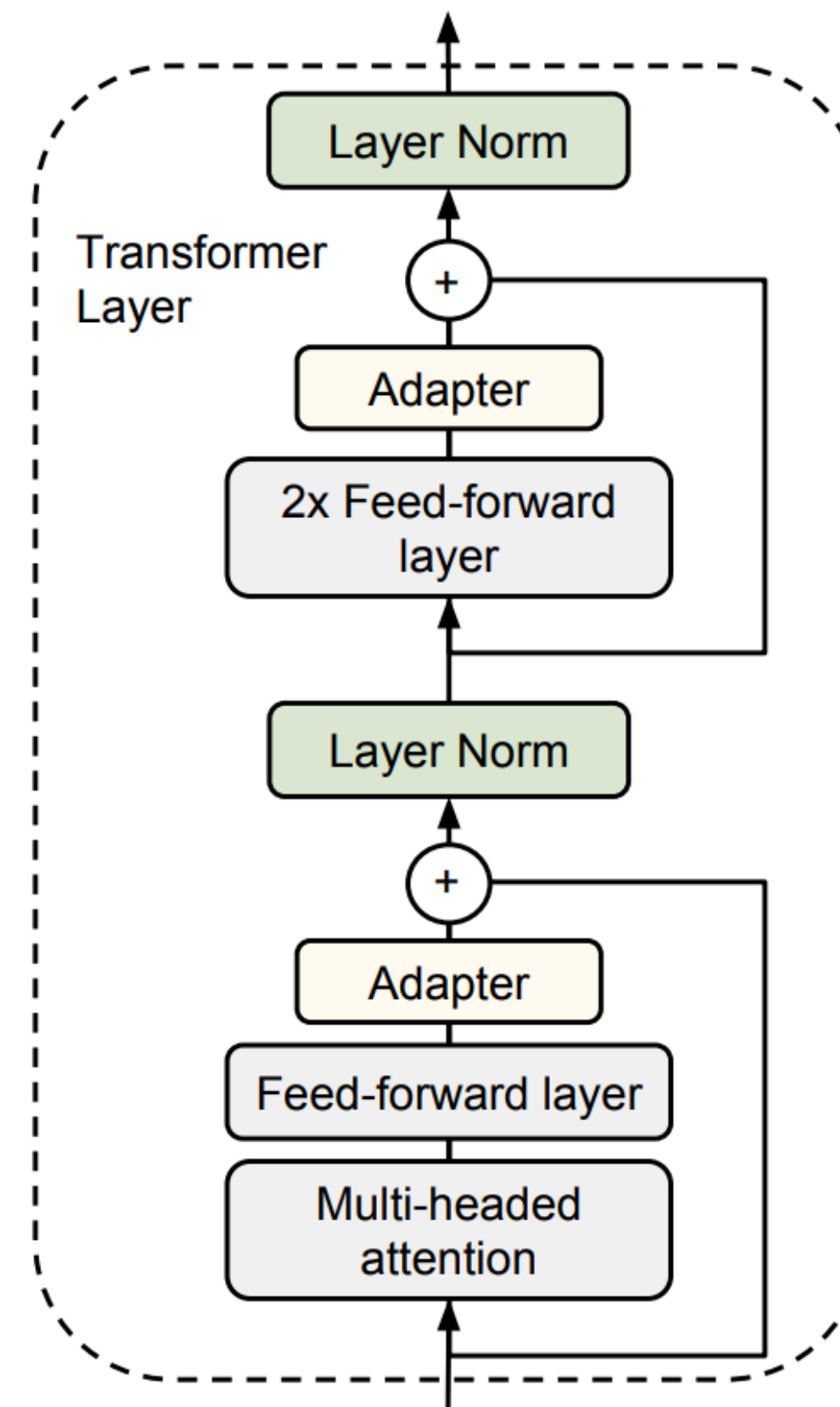


Prompt Tuning: Disadvantages

- Addition of a trained prompt limits the maximum length and slows down the model
- Prompt Tuning is very unstable and quality changes non-monotonically as the size of the prompt and model increases

Adapters

- After each attention layer and FFN, a **small** trainable adapter is added
- The adapter has skip-connection and two full-connection layers with dimensionality reduction (reduces the number of parameters)
- Only the adapters, normalizations and head are trained
- Such technique reaches fine-tuning in terms of quality



Adapters: Disadvantages

- The basic version of the adapter adds quite a few parameters compared to Prompt Tuning
- Adapters add extra layers that cannot be computed in parallel
- This slows down the model. Especially for small batch sizes

Intrinsic Dimensionality of Language Models

Only a small part of the parameters is required for solving a downstream task.

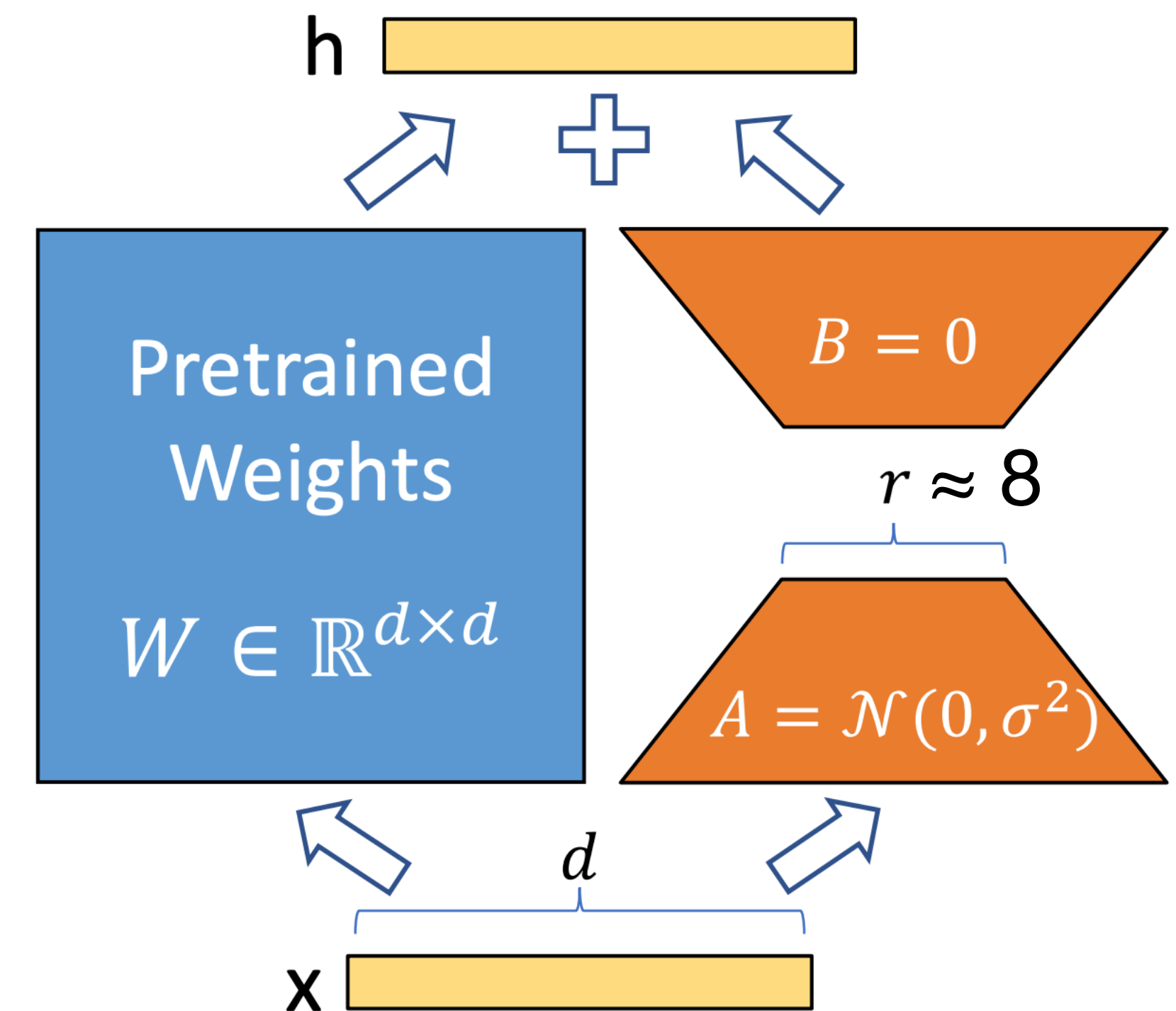


LoRA

Low-Rank Adaptation

- **Idea:** To adapt the model to the new task, we need to shift the weights towards the anti-gradient direction

$$W' = W + \delta W$$



LoRA

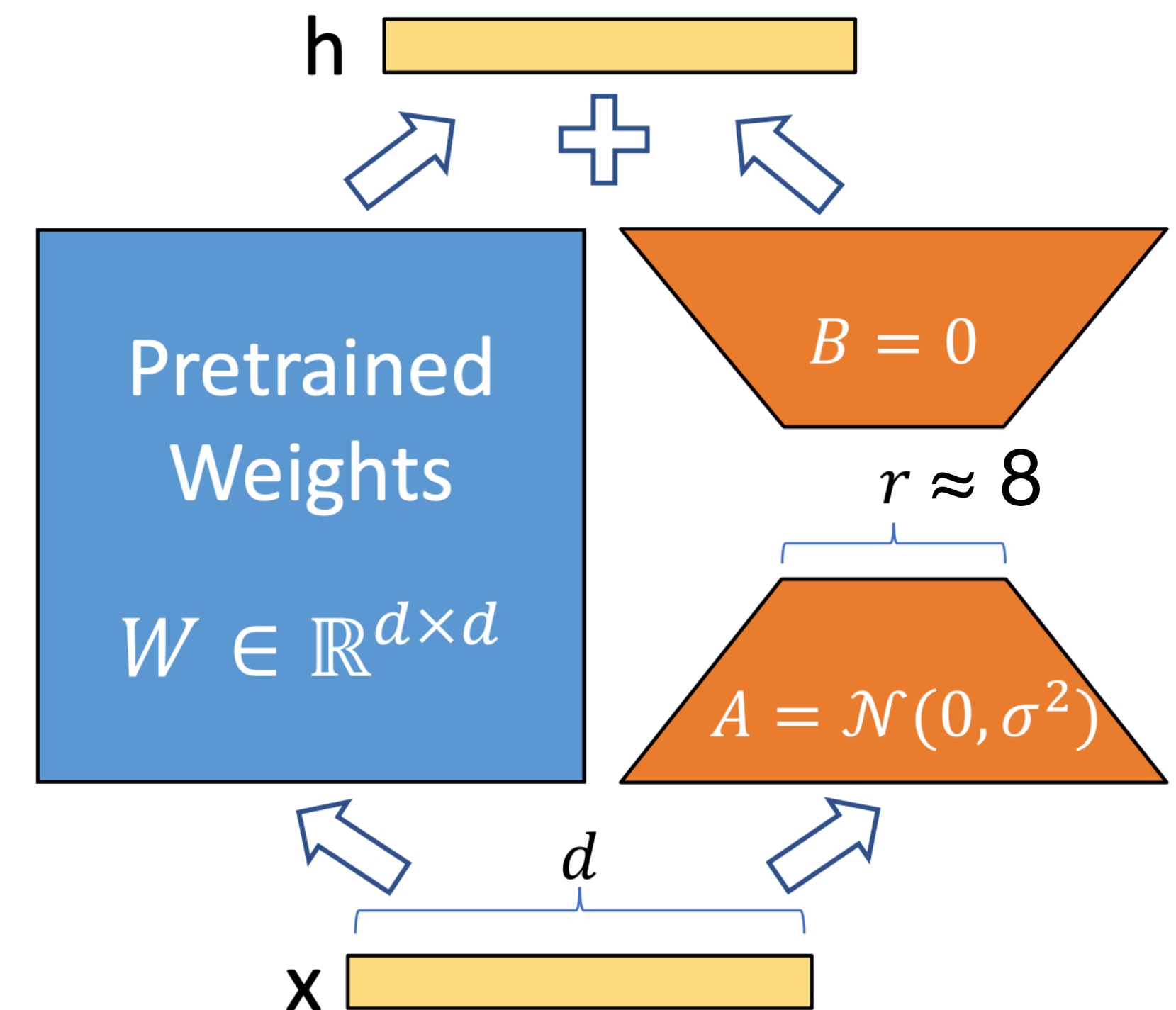
Low-Rank Adaptation

- **Idea:** To adapt the model to the new task, we need to shift the weights towards the anti-gradient direction

$$W' = W + \delta W$$

- We approximate δW by the product of the training matrices AB

$$W' = W + AB$$



LoRA

Low-Rank Adaptation

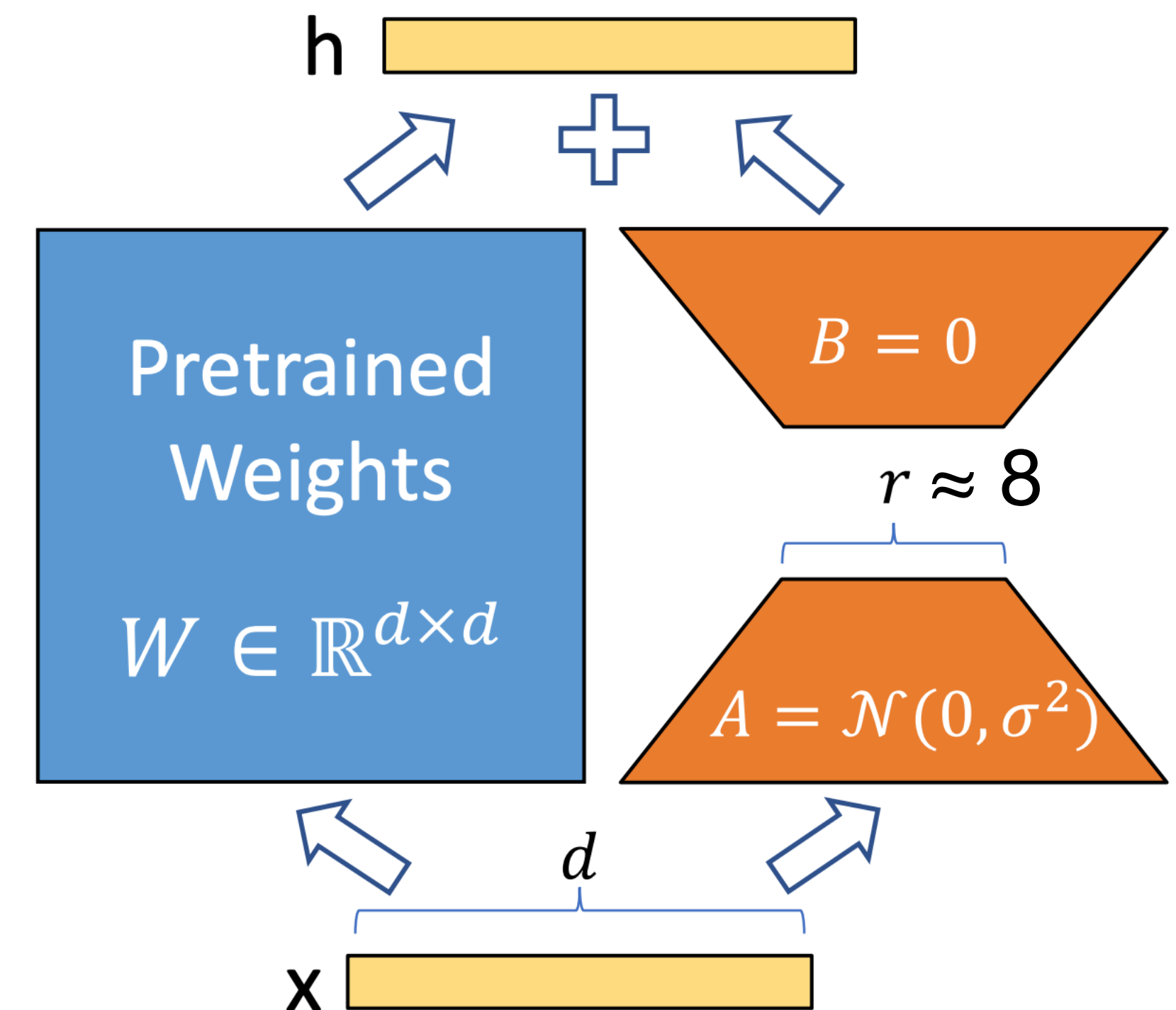
- **Idea:** To adapt the model to the new task, we need to shift the weights towards the anti-gradient direction

$$W' = W + \delta W$$

- We approximate δW by the product of the training matrices AB

$$W' = W + AB$$

- Only the matrices W_q and W_v of the attention mechanism are changed in this way
- This way very few parameters are added and the addition can be considered in parallel with the main block



LoRA

Low-Rank Adaptation

The change on r have a marginal impact on quality.

	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL($\pm 0.5\%$)	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI ($\pm 0.1\%$)	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

BitFit

Bias-terms Fine-tuning

Idea: biases have very few parameters, we will train only them

Attention

$$\mathbf{Q}^{m,\ell}(\mathbf{x}) = \mathbf{W}_q^{m,\ell} \mathbf{x} + \mathbf{b}_q^{m,\ell}$$

$$\mathbf{K}^{m,\ell}(\mathbf{x}) = \mathbf{W}_k^{m,\ell} \mathbf{x} + \mathbf{b}_k^{m,\ell}$$

$$\mathbf{V}^{m,\ell}(\mathbf{x}) = \mathbf{W}_v^{m,\ell} \mathbf{x} + \mathbf{b}_v^{m,\ell}$$

$$\mathbf{h}_1^\ell = \text{att}(\mathbf{Q}^{1,\ell}, \mathbf{K}^{1,\ell}, \mathbf{V}^{1,\ell}, \dots, \mathbf{Q}^{m,\ell}, \mathbf{K}^{m,\ell}, \mathbf{V}^{m,\ell})$$

Feed Forward Network

$$\mathbf{h}_2^\ell = \text{Dropout}(\mathbf{W}_{m_1}^\ell \cdot \mathbf{h}_1^\ell + \mathbf{b}_{m_1}^\ell)$$

$$\mathbf{h}_3^\ell = \mathbf{g}_{LN_1}^\ell \odot \frac{(\mathbf{h}_2^\ell + \mathbf{x}) - \mu}{\sigma} + \mathbf{b}_{LN_1}^\ell$$

$$\mathbf{h}_4^\ell = \text{GELU}(\mathbf{W}_{m_2}^\ell \cdot \mathbf{h}_3^\ell + \mathbf{b}_{m_2}^\ell)$$

$$\mathbf{h}_5^\ell = \text{Dropout}(\mathbf{W}_{m_3}^\ell \cdot \mathbf{h}_4^\ell + \mathbf{b}_{m_3}^\ell)$$

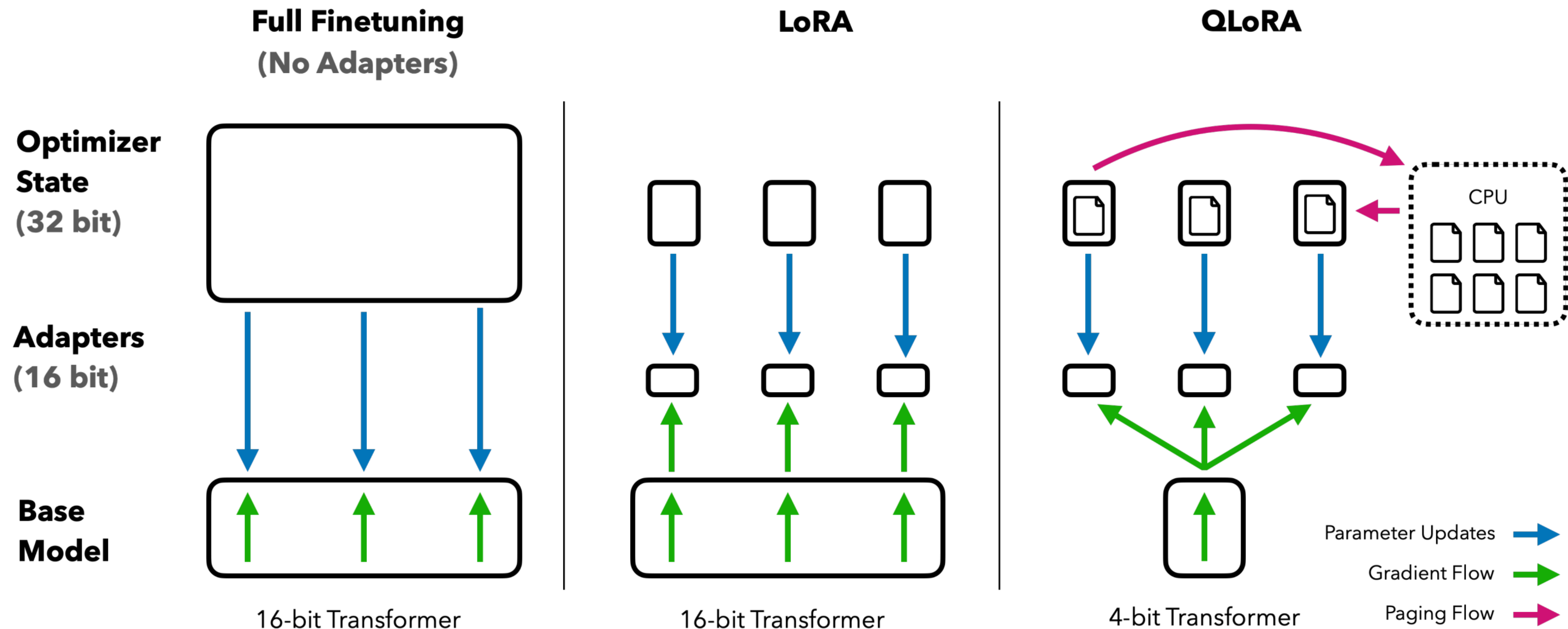
$$\text{out}^\ell = \mathbf{g}_{LN_2}^\ell \odot \frac{(\mathbf{h}_5^\ell + \mathbf{h}_3^\ell) - \mu}{\sigma} + \mathbf{b}_{LN_2}^\ell$$

Comparison of methods

Model&Method	# Trainable Parameters	WikiSQL	MNLI-m	SAMSum
		Acc. (%)	Acc. (%)	R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

QLoRA

Designed for tuning very large models.



$$\mathbf{Y}^{\text{BF16}} = \mathbf{X}^{\text{BF16}} \text{doubleDequant}(c_1^{\text{FP32}}, c_2^{\text{k-bit}}, \mathbf{W}^{\text{NF4}}) + \mathbf{X}^{\text{BF16}} \mathbf{L}_1^{\text{BF16}} \mathbf{L}_2^{\text{BF16}}$$